CSC 2032 – Object Oriented Programming

Unified Student Dashboard System

Submitted By:

- AS20240536 – R.M.P.G. Rathnaweera
- AS20240389 – K.D.N. Karannagoda

Table of Contents

1. Introduction

1.1. Background

Modern university students navigate a demanding academic environment that extends beyond traditional coursework. A student's week involves balancing academic submissions, health and household chores, and leadership or active duties in campus clubs and extra-curricular organizations. Managing these diverse responsibilities effectively requires structured organization, clear prioritization, and proactive dynamic planning.

1.2 Problem Statement

Juggling university assignments, personal chores, and club/work responsibilities across three different platforms or applications (such as digital calendars, stand-alone to-do lists, and separate note-taking tools) causes critical tasks to slip through the cracks. Existing applications isolate tasks into fragmented sets, treating all to-dos uniformly regardless of their contextual urgency.

1.3 Objectives

The primary objective of this project is to develop an Object-Oriented Unified Student Dashboard in Java that provides:

• Centralized Management: Managing academic assignments, personal chores, and club responsibilities under a single interface.

• Intelligent Prioritization: Implementing an algorithmic 'Urgency Score' engine based on remaining time to deadline.

• Modularity and Maintainability: Utilizing fundamental Object-Oriented Programming (OOP) paradigms (Encapsulation, Inheritance, Polymorphism, Abstraction) for long-term scalability.

1.4 Scope

The scope of this project includes system architecture, relational database modeling, core algorithmic design, and Object-Oriented implementation in Java. The solution focuses on task entry, specialized task categorization (Assignment, Personal, Clubs), dynamic score recalculation and structured data displaying (Urgency Score Dashboard).

2. System Overview

2.1 Proposed Solution

The proposed Unified Student Dashboard aggregates all student commitments into a single streamlined pipeline. Rather than simply presenting a flat list of tasks, the dashboard dynamically evaluates and sorts active tasks using a composite mathematical Urgency Score algorithm. This empowers students to focus immediately on high-impact, impending obligations.

2.2 User Roles

• Student (User): Enters and manages tasks, configures task parameter as deadline, triggers dynamic sorting, and marks tasks as completed or snoozed (For personal).
• System (Automated Core): Handles background score recalculations, displays tasks to an urgency order, and enforces data integrity rules.

2.3 Use Case Diagram

The system's interactions are structured around two key actors: the User (Student) and the automated System backend. Below is the textual representation of the Use Case interaction flow derived from the reference architecture:

Use Case ID	Use Case Name	Included / Extended Interactions
UC-01	Create / Edit Task	Includes <<Define task parameters>>
UC-02	Mark Task Complete	Extends <<Archive completed tasks>>
UC-03	Recalculate urgency score dynamically	-
UC-04	View Prioritized Dashboard	Includes <<Calculate urgency score>>, <<Sort active tasks>>

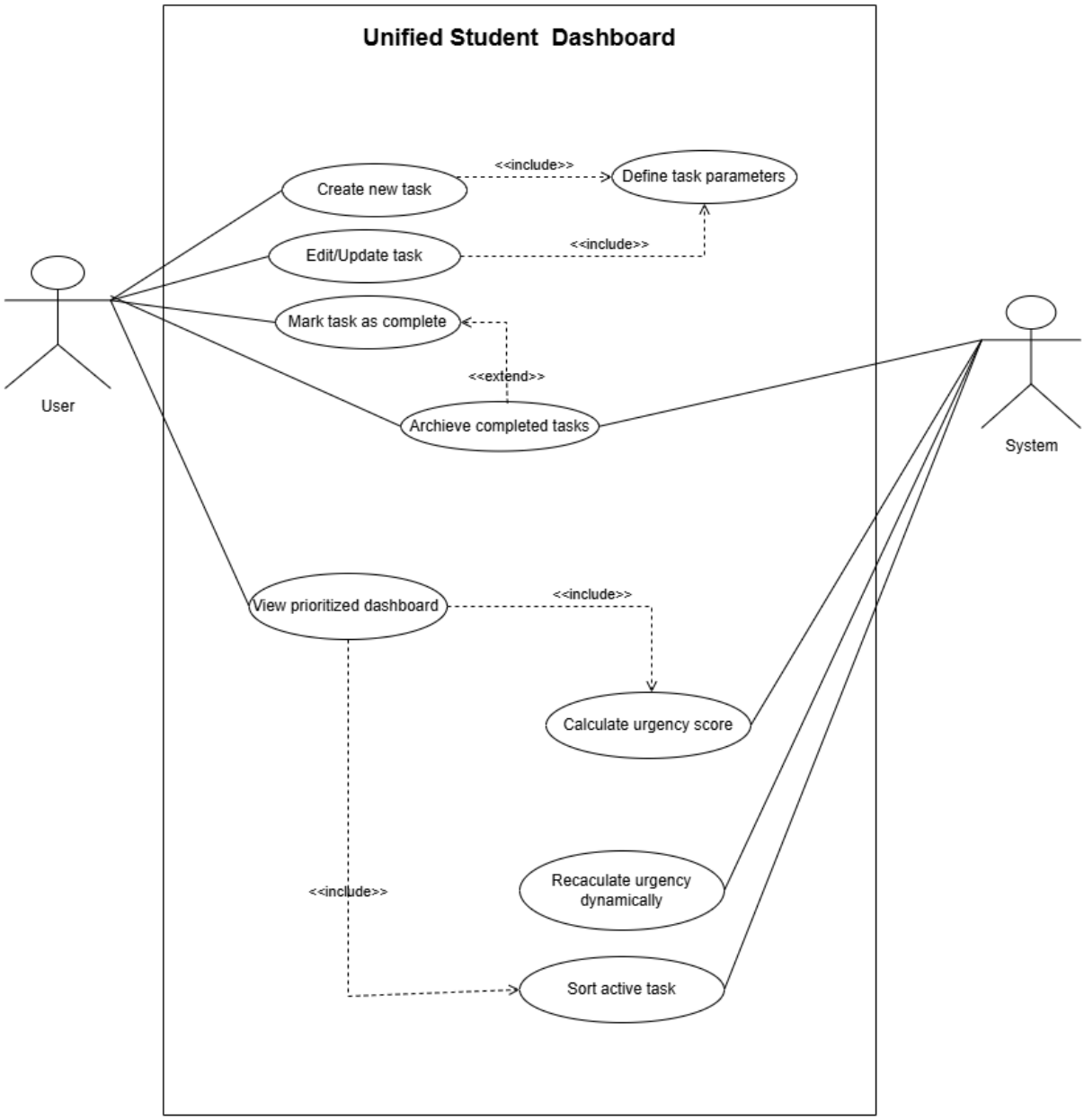
Unified Student Dashboard
Create new task
<<include>>
Define task parameters
Edit/Update task
<<include>>
Mark task as complete
<<extend>>
Archieve completed tasks
User
System
View prioritized dashboard
<<include>>
Calculate urgency score
<<include>>
Recaculate urgency dynamically
Sort active task

Figure 1

3. Object-Oriented Design

3.1 Identifying Classes and Responsibilities

The application design follows clean domain-driven architecture.

Class Name	Inheritance / Role	Core Responsibilities
Student	Superclass	Defines student attributes (studentId, studentName, emai,password)
Task	Superclass	Composition with student
Assignment	Subclass of Task	Extends Task with academic-specific properties like subjectName.
Clubs	Subclass of Task	Extends Task with clubName and responsibilityType
Personal	Subclass of Task	Extends Task with description, isRecurring , and snooze.
Urgency Profile	Composition with Task	Composition with task.
Urgency Profile Dashboard	-	Uses urgencyProfile and student

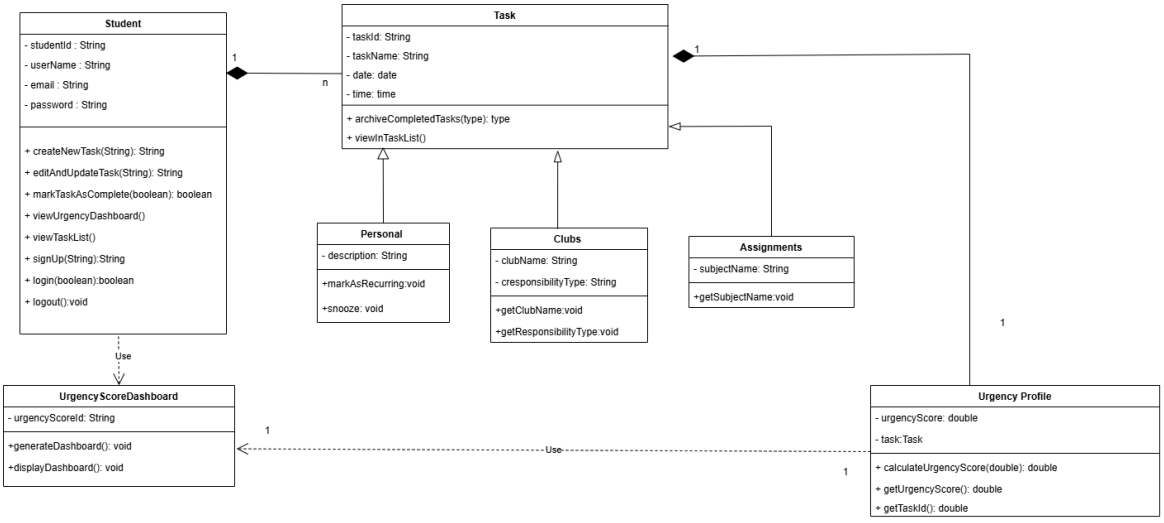

Figure 2

3.2 Applying Core OOP Principles & sample code snippets

3.2.1 Encapsulation

All member variables in entities (e.g., taskId, taskName, date, time) are marked as private. Access and state mutations are strictly controlled through public getters and validation-enforcing setters, ensuring data integrity across the system.

```java
public class UrgencyProfile {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private double estimatedEffortInHours;
    private double weight;
    private double urgencyScore;
    private double remainingWorkload;
    private LocalDate deadline;

    @OneToOne(mappedBy = "urgencyProfile")
    private Task task;

    @OneToOne(mappedBy = "urgencyProfile", cascade = CascadeType.ALL)
    private UrgencyScoreDashboard dashboard;

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public double getEstimatedEffortInHours() { return estimatedEffortInHours; }
    public void setEstimatedEffortInHours(double estimatedEffortInHours) { this.estimatedEffortInHours = estimatedEffortInHours; }

    public double getWeight() { return weight; }
    public void setWeight(double weight) { this.weight = weight; }

    public double getUrgencyScore() { return urgencyScore; }
    public void setUrgencyScore(double urgencyScore) { this.urgencyScore = urgencyScore; }

    public double getRemainingWorkload() { return remainingWorkload; }
    public void setRemainingWorkload(double remainingWorkload) { this.remainingWorkload = remainingWorkload; }

    public LocalDate getDeadline() { return deadline; }
    public void setDeadline(LocalDate deadline) { this.deadline = deadline; }

    public Task getTask() { return task; }
    public void setTask(Task task) { this.task = task; }

    public UrgencyScoreDashboard getDashboard() { return dashboard; }
    public void setDashboard(UrgencyScoreDashboard dashboard) { this.dashboard = dashboard; }
}
```

Figure 3

3.2.2 Inheritance

Common fields and base behavior are localized in the abstract base class Task. Specific task variants—Assignment, Clubs, and Personal—inherit from Task, avoiding code duplication and establishing an IS-A relationship.

```
p C:\Users\ASUS TUF\Downloads\student_dashboard\demo\src\main

import jakarta.persistence.*;
import java.time.LocalDate;
import java.time.LocalTime;
import com.fasterxml.jackson.annotation.JsonSubTypes;
import com.fasterxml.jackson.annotation.JsonTypeInfo;

@Entity
@Table(name = "Task")
@Inheritance(strategy = InheritanceType.JOINED)
@JsonTypeInfo(
    use = JsonTypeInfo.Id.NAME,
    include = JsonTypeInfo.As.EXISTING_PROPERTY,
    property = "taskType",
    visible = true
)
@JsonSubTypes({
    @JsonSubTypes.Type(value = Assignment.class, name = "Assignment"),
    @JsonSubTypes.Type(value = Personal.class, name = "Personal"),
    @JsonSubTypes.Type(value = Clubs.class, name = "Clubs")
})
public abstract class Task {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int taskId;

    private String taskName;
    private LocalDate date;
    private LocalTime time;

    private String taskType;
    private boolean isCompleted = false;

```

Figure 4 - Task Super class (inheritance)

```java
    @ManyToOne
    @JoinColumn(name = "studentId", nullable = false)
    private Student student;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "urgencyProfileId")
    private UrgencyProfile urgencyProfile;

    // Getters and Setters
    public int getTaskId() { return taskId; }
    public void setTaskId(int taskId) { this.taskId = taskId; }

    public String getTaskName() { return taskName; }
    public void setTaskName(String taskName) { this.taskName = taskName; }

    public LocalDate getDate() { return date; }
    public void setDate(LocalDate date) { this.date = date; }

    public LocalTime getTime() { return time; }
    public void setTime(LocalTime time) { this.time = time; }

    public String getTaskType() { return taskType; }
    public void setTaskType(String taskType) { this.taskType = taskType; }

    public Student getStudent() { return student; }
    public void setStudent(Student student) { this.student = student; }

    public UrgencyProfile getUrgencyProfile() { return urgencyProfile; }
    public void setUrgencyProfile(UrgencyProfile urgencyProfile) { this.urgencyProfile = urgencyProfile; }

    public boolean getIsCompleted() { return isCompleted; }
    public void setIsCompleted(boolean isCompleted) { this.isCompleted = isCompleted; }
}
```

Figure 5 - Task Super class 2 (inheritance)

```java
package com.dashboard.model;

import jakarta.persistence.Entity;
import jakarta.persistence.Table;

@Entity
@Table(name = "Assignment")
public class Assignment extends Task {

    private String subjectName;

    public String getSubjectName() {
        return subjectName;
    }

    public void setSubjectName(String subjectName) {
        this.subjectName = subjectName;
    }
}
```

Figure 6 - Assignment sub class (inheritance)

```java
package com.dashboard.model;

import jakarta.persistence.Entity;
import jakarta.persistence.Table;

@Entity
@Table(name = "Personal")
public class Personal extends Task {

    private String description;


    private boolean isRecurring;
    private boolean snooze;

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public boolean isRecurring() {
        return isRecurring;
    }

    public void setRecurring(boolean isRecurring) {
        this.isRecurring = isRecurring;
    }

    public boolean isSnooze() {
        return snooze;
    }

    public void setSnooze(boolean snooze) {
        this.snooze = snooze;
    }
}
```

Figure 7 - Personal sub class (inheritance)

```java
package com.dashboard.model;

import jakarta.persistence.Entity;
import jakarta.persistence.Table;

@Entity
@Table(name = "Clubs")
public class Clubs extends Task {

    private String clubName;
    private String responsibilityType;

    public String getClubName() {
        return clubName;
    }

    public void setClubName(String clubName) {
        this.clubName = clubName;
    }

    public String getResponsibilityType() {
        return responsibilityType;
    }

    public void setResponsibilityType(String responsibilityType) {
        this.responsibilityType = responsibilityType;
    }
}
```

Figure 8 - Clubs sub class (inheritance)

3.2.3 Polymorphism

Polymorphism is demonstrated through the taskController() method. Each subclass implements custom scoring logic: Assignment factors in Clubs evaluates event proximity and role level, while Personal considers snooze count and recurrence intervals. A collection of List<Task> can be processed polymorphically without knowing concrete types at compile-time.

```java
package com.dashboard.controller;

import com.dashboard.model.Task;
import com.dashboard.service.TaskService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import org.springframework.http.ResponseEntity;
import java.util.List;

@RestController
@RequestMapping("/api/tasks")
@CrossOrigin(origins = "*")
public class TaskController {

    @Autowired
    private TaskService taskService;


    @PostMapping("/add")
    public Task addTask(@RequestBody Task task) {
        return taskService.saveTask(task);
    }


    @PostMapping("/{id}/complete")
    public ResponseEntity<?> markTaskAsComplete(@PathVariable int id) {
        taskService.markTaskAsComplete(id);
        return ResponseEntity.ok().build();
    }


    @DeleteMapping("/{id}")
    public ResponseEntity<?> deleteTask(@PathVariable int id) {
        taskService.deleteTask(id);
        return ResponseEntity.ok().build();
    }


    @GetMapping("/student/{studentId}")
    public List<Task> getTasksByStudentId(@PathVariable int studentId) {
        return taskService.getTasksByStudentId(studentId);
    }
}
```

Figure 9

3.2.4 Abstraction

Abstraction is achieved by hiding complex calculation algorithms, persistence implementations, and urgency engine operations behind clean interface definitions.

C:\Users\ASUS TUF\Downloads\student_dashboard\demo\src\main

```java
import jakarta.persistence.*;
import java.time.LocalDate;
import java.time.LocalTime;
import com.fasterxml.jackson.annotation.JsonSubTypes;
import com.fasterxml.jackson.annotation.JsonTypeInfo;

@Entity
@Table(name = "Task")
@Inheritance(strategy = InheritanceType.JOINED)
@JsonTypeInfo(
    use = JsonTypeInfo.Id.NAME,
    include = JsonTypeInfo.As.EXISTING_PROPERTY,
    property = "taskType",
    visible = true
)
@JsonSubTypes({
    @JsonSubTypes.Type(value = Assignment.class, name = "Assignment"),
    @JsonSubTypes.Type(value = Personal.class, name = "Personal"),
    @JsonSubTypes.Type(value = Clubs.class, name = "Clubs")
})
public abstract class Task {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int taskId;

    private String taskName;
    private LocalDate date;
    private LocalTime time;

    private String taskType;
    private boolean isCompleted = false;

```

Figure 10 - Abstraction

```java
    @ManyToOne
    @JoinColumn(name = "studentId", nullable = false)
    private Student student;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "urgencyProfileId")
    private UrgencyProfile urgencyProfile;

    // Getters and Setters
    public int getTaskId() { return taskId; }
    public void setTaskId(int taskId) { this.taskId = taskId; }

    public String getTaskName() { return taskName; }
    public void setTaskName(String taskName) { this.taskName = taskName; }

    public LocalDate getDate() { return date; }
    public void setDate(LocalDate date) { this.date = date; }

    public LocalTime getTime() { return time; }
    public void setTime(LocalTime time) { this.time = time; }

    public String getTaskType() { return taskType; }
    public void setTaskType(String taskType) { this.taskType = taskType; }

    public Student getStudent() { return student; }
    public void setStudent(Student student) { this.student = student; }

    public UrgencyProfile getUrgencyProfile() { return urgencyProfile; }
    public void setUrgencyProfile(UrgencyProfile urgencyProfile) { this.urgencyProfile = urgencyProfile; }

    public boolean getIsCompleted() { return isCompleted; }
    public void setIsCompleted(boolean isCompleted) { this.isCompleted = isCompleted; }
}
```

Figure 11 - Abstraction 2

3.3 Dynamic Urgency Scoring Engine Formula

The core innovation of the dashboard is the dynamic score calculation formula. The score is computed using three primary parameters:

1. Time remaining (T): Hours until the target deadline.
2. Estimated effort in hours (E): Estimated hours needed to complete the task.
3. Weight (W): Scalar value representing importance.

Mathematical Formulation:

Urgency Score = (Weight * Estimated effort in Hours)/ (Time remaining)

If Time remaining = 0 (overdue), Urgency Score defaults to maximum critical priority (10/10).

4. Database Design

4.1 Entity-Relationship Diagram & Relational Schema Analysis

The database architecture utilizes a specialized generalization/specialization relational design (Class Hierarchy Table Pattern). The central table 'task' holds common fields, while child tables ('assignment', 'clubs', 'personal') store subtype-specific fields, linked via taskId foreign keys.

4.2 Table Summary

Table Name	Primary Key	Foreign Keys	Key Fields & Types
student	studentId (INT)	None	userName VARCHAR(50), email VARCHAR(100), password VARCHAR(255)
task	taskId (INT)	studentId	taskName VARCHAR(100), date DATE, time TIME, taskType ENUM
assignment	taskId (INT)	taskId	subjectName VARCHAR(100)
clubs	taskId (INT)	taskId	clubName VARCHAR(100), responsibilityType VARCHAR(100)
personal	taskId (INT)	taskId	description VARCHAR(255), isRecurring TINYINT(), snooze TINYINT()

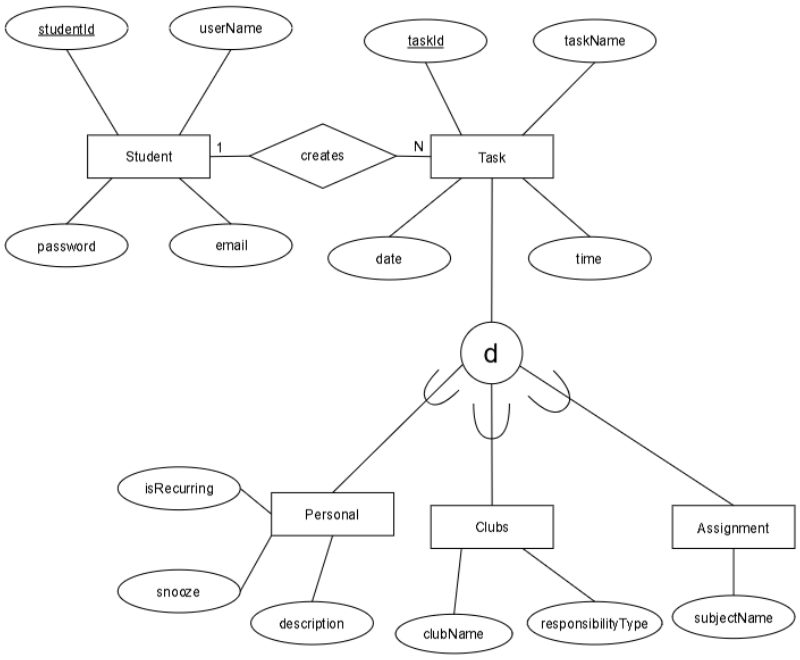

Figure 12 - ER diagram

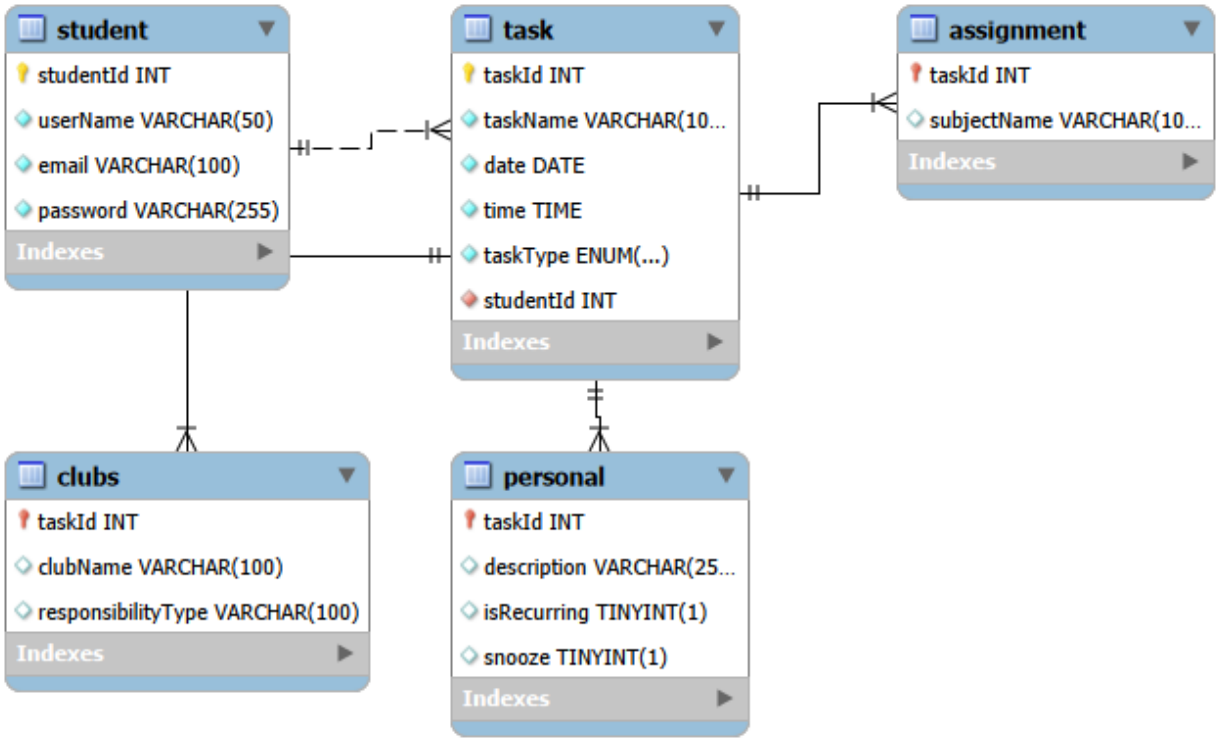

Figure 13 - Relational schema

5.System Architecture

5.1 Suggested Package Structure

To adhere to clean architectural principles and Separation of Concerns (SoC), the Java project is organized into clear packages:

```
com.dashboard.student
├─── controller
│    ├─── StudentController.java
│    ├─── TaskController.java
│    └─── WebController.java
├─── model
│    ├─── Assignment.java
│    ├─── Clubs.java
│    ├─── Personal.java
│    ├─── Student.java
│    ├─── Task.java
│    ├─── UrgencyProfile.java
│    └─── UrgencyScoreDashboard.java
├─── repository
│    ├─── StudentRepository.java
│    └─── TaskRepository.java
├─── service
│    ├─── StudentService.java
│    ├─── TaskService.java
│    ├─── UrgencyCalculator.java
│    └─── UrgencyCalculatorImpl.java
└─── DemoApplication.java
```

Figure 14

6. Testing Plan

The application will undergo comprehensive testing phases:

1. Unit Testing: Testing individual class methods.
2. Integration Testing: Verifying data persistence and extraction from the MySQL database.
3. System Testing: Validating user workflow from task creation to dynamic dashboard updating.

7. Future Enhancements

• Notification System: Adding a notification system will encourage students to do their tasks without being late.
• Google Calendar API Integration: Bi-directional synchronization of personal chores and club events.
• AI-Assisted Time Estimation: Employing machine learning models to adjust estimated task effort based on past student performance.

8. Conclusion

The Unified Student Dashboard provides a great solution to the challenge of multi-app fragmentation. By applying Object-Oriented software engineering principles in Java, the system offers a scalable, modular architecture that prioritizes student responsibilities through a dynamic urgency algorithm, ensuring academic success and reduced cognitive overload.

THANK YOU